République Algérienne Démocratique et Populaire

Ministère de l'enseignement supérieur et la recherche scientifique

Université d'Alger 1 | **جامعة الجزائر 01**

Faculté des Science | **كلية العلوم**

Département informatique | **قسم الإعلام الآلي**



Image processing

mini-Project

Report

Presented by :

BAZINE Rafie
Berraki zineddine
Hadil Cheikh
Saidani Abderraouf
Haddab Selsabil

Introduction



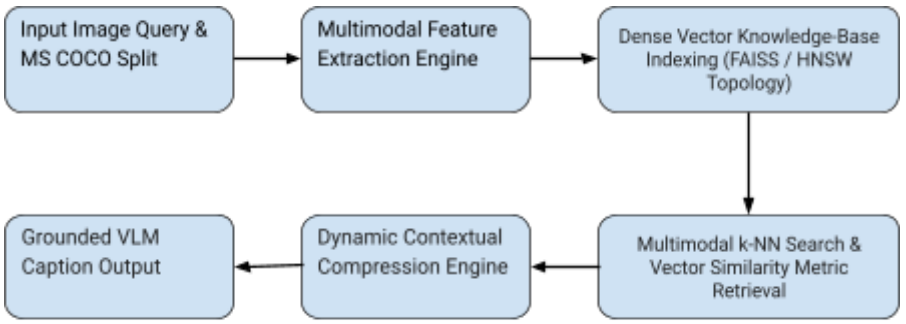
Imagine asking a Vision-Language Model (VLM) to describe a quiet office scene, only for it to confidently report a sleeping cat on the desk or completely miss the laptop resting right in front of the window. Why do cutting-edge models suffer from these frustrating **object hallucinations** and **object omissions**?

The root cause lies in visual ambiguity and strong statistical priors learned during training. To eliminate these flaws, we turn to **Retrieval-Augmented Generation (RAG)** : grounding generation in external, verified multimodal context. But a RAG system is only as reliable as the data pipeline supporting it.
In this project, we want to master the technical architecture required to build a bulletproof multimodal data pipeline on the MS COCO dataset, ensuring absolute experimental alignment and robust hybrid index design. Our goal is to:

- **Design** an end-to-end data acquisition and candidate integrity verification pipeline for MS COCO archives.
- **Implement** an 8-bucket stratified sampling engine with dynamic fallback reallocation to resolve candidate starvation pools.
- **Enforce** strict zero-leakage data isolation across knowledge base and evaluation subsets using programmatic assertion audits.
- **Translate** statistical Exploratory Data Analysis (EDA) insights into a dual-index (dense visual + sparse lexical) RAG retrieval strategy.

Architectural Overview & Contextual Compression RAG Strategy

The primary technical mission of the `rag-image-captioning` architecture is to engineer an enterprise-grade Contextual Compression Retrieval-Augmented Generation (RAG) framework tailored for automated image captioning on the MS COCO (Microsoft Common Objects in Context) dataset. Standard Vision-Language Models (VLMs) frequently suffer from object hallucinations, high computational overhead, and semantic dilution caused by processing uncompressed multimodal contexts. By establishing a dynamic contextual compression retrieval engine, this system grounds VLM generation in verified visual and textual metadata. Dynamic context compression selectively filters out visual redundancy and noisy text tokens at the embedding and prompt interface level. This lowers object hallucination rates, minimizes cross-modal inference latency, and significantly increases the metric-level precision (BLEU-4, METEOR, ROUGE-L, CIDEr, SPICE) of generated captions.



RAG ARCHITECTURE PIPELINE

Phase 1 :

Data Exploration & Preprocessing

1- Data Acquisition & Integrity Verification Architecture

Before constructing retrieval indexes or evaluating VLM outputs, we must establish an automated, deterministic ingestion pipeline. Raw dataset archives are notoriously prone to corruption, missing files, and variable annotation structures. Our goal is to convert raw MS COCO 2017 archives into a verified, uniform candidate pool.

The pipeline begins by acquiring three core MS COCO archives:

- `annotations_trainval2017.zip` (241.19 MB): Contains raw instance bounding boxes and human-annotated captions.
- `train2017.zip` (18.01 GB): The primary source for our multimodal knowledge base and validation splits.
- `val2017.zip` (777.80 MB): The foundational pool for our isolated evaluation benchmarks.

2- Candidate Integrity Verification Filters

Once extracted, we passed two rigorous verification checks to prevent runtime failures and evaluation bias:

1. **Physical Disk Integrity Check:** Every image ID recorded in the annotation JSON must programmatically resolve to an existing, non-corrupted JPEG file on disk. If an annotation reference lacks a corresponding image file, it is immediately discarded.
2. **Caption Volume Truncation Filter:** Standard evaluation metrics (such as BLEU, METEOR, CIDEr, and SPICE) require uniform reference depth. Candidate images must possess at least **5 annotated human reference captions**. To maintain metric consistency across all splits, excess captions are truncated to exactly 5 per image.

When executed against the `val2017` dataset split, this automated verification pipeline filtered out incomplete instances and isolated exactly **5,000 valid candidate images** ready for structured sampling.

3- Multi-Dimensional Stratification & Dynamic Fallback Reallocation

Simple random sampling is the enemy of reliable model evaluation. If a test set accidentally skew heavily toward simple, single-object scenes, VLM evaluation metrics will falsely reflect high performance while masking severe hallucinations in cluttered environments.

To prevent this, we constructed an **8-Bucket Stratified Sampling Engine** that partitions candidate images across three orthogonal visual dimensions:

- **Scene Complexity:** Categorized via bounding-box instance counts into **Simple** ($1 \leq objects \leq 2$) versus **Complex** ($objects \geq 3$).

```
================================================
   Scene Complexity Analysis: Test Subset (test_100.json)
================================================
Total Images Analyzed: 100
------------------------------------------------
📊 Categorical Breakdown:
  • Simple Scenes (1-2 objects): 52    (52.00%)
  • Complex Scenes (>=3 objects): 48    (48.00%)
------------------------------------------------
📈 Statistical Metrics:
  • Mean (Average) Objects / Image: 5.41
  • Median Objects / Image:        2.00
  • Min Objects in an Image:       1
  • Max Objects in an Image:       56
```

```
================================================
   Scene Complexity Analysis: Test Subset (test_1000.json)
================================================
Total Images Analyzed: 1000
------------------------------------------------
📊 Categorical Breakdown:
  • Simple Scenes (1-2 objects): 500   (50.00%)
  • Complex Scenes (>=3 objects): 500   (50.00%)
------------------------------------------------
📈 Statistical Metrics:
  • Mean (Average) Objects / Image: 5.67
  • Median Objects / Image:        2.50
  • Min Objects in an Image:       1
  • Max Objects in an Image:       41
```

```
================================================
   Scene Complexity Analysis: Validation Subset (val_100.json)
================================================
Total Images Analyzed: 100
------------------------------------------------
📊 Categorical Breakdown:
  • Simple Scenes (1-2 objects): 30    (30.00%)
  • Complex Scenes (>=3 objects): 68    (68.00%)
  • Unannotated Scenes (0 objects): 2
------------------------------------------------
📈 Statistical Metrics:
  • Mean (Average) Objects / Image: 7.48
  • Median Objects / Image:        4.00
  • Min Objects in an Image:       0
  • Max Objects in an Image:       36
```

```
================================================
   Scene Complexity Analysis: Knowledge Base Subset (kb_10k.json)
================================================
Total Images Analyzed: 10000
------------------------------------------------
📊 Categorical Breakdown:
  • Simple Scenes (1-2 objects): 3484  (34.84%)
  • Complex Scenes (>=3 objects): 6297  (62.97%)
  • Unannotated Scenes (0 objects): 219
------------------------------------------------
📈 Statistical Metrics:
  • Mean (Average) Objects / Image: 6.71
  • Median Objects / Image:        4.00
  • Min Objects in an Image:       0
  • Max Objects in an Image:       56
```

- **Object Interaction:** Categorized into **Non-Interactive** (1 distinct object category) versus **Interactive** ($\geq$ 2 distinct object categories).
- **Environmental Context:** Classified into **Indoor** vs. **Outdoor** environments using majority voting over COCO object supercategories (e.g., mapping `furniture` and `appliance` to Indoor, `vehicle` and `animal` to Outdoor).

4 - The Candidate Starvation Challenge & Dynamic Fallback

When partitioning candidates into structural splits, targeted allocations require equal representation across all 8 buckets. For instance, creating `test_1000.json` requires 125 images per bucket. However, real-world data distributions are naturally imbalanced.

In our dataset, **Bucket 5** (Complex, Non-Interactive, Indoor) contained only 19 initial candidates. After `test_100.json` consumed 12 images, Bucket 5 was left with a severe **candidate starvation pool** of only 7 remaining images—creating an **118-image deficit** for `test_1000.json`.

To solve this without breaking experimental integrity, we implement a **Dynamic Fallback Reallocation Algorithm** using a fixed random seed (`seed = 42`):

1. The engine detects the candidate deficit in starved buckets.
2. It dynamically reallocates the missing sample quota to eligible excess buckets holding the largest remaining candidate pools (primarily **Bucket 7**: Complex, Interactive, Indoor and **Bucket 8**: Complex, Interactive, Outdoor).
3. This guarantees that `test_1000.json` successfully reaches its 1,000-image capacity while preserving maximum structural clutter complexity

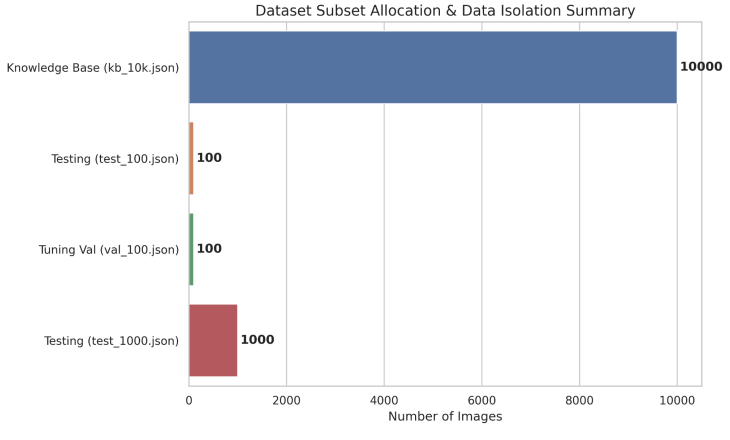
5- Zero-Leakage Isolation & Data Governance

In Retrieval-Augmented Generation, data leakage is a catastrophic flaw. If an image or caption evaluated in the test set is present inside the RAG retrieval knowledge base, the system will perform lookup rather than true context-grounded generation, invalidating the research experiment.

To enforce strict scientific validity, we establish the **Anti-Leakage Rule**: Reference captions and image IDs reserved for testing and validation must *never* exist within the knowledge base or be retrieved for generation.

6 - Dataset Partitioning Summary

We partition our curated data into four strict sub-datasets:



- `test_100.json` (100 images / 500 captions): Small-scale rapid benchmark sourced from `val2017`.
- `test_1000.json` (1,000 images / 5,000 captions): Large-scale formal benchmark sourced from `val2017`.
- `val_100.json` (100 images / 500 captions): Hyperparameter tuning validation set sourced from `train2017`.
- `kb_10k.json` (10,000 images / 50,000 captions): The central RAG multimodal knowledge base sourced from `train2017`.

7- Programmatic Leakage Isolation Audits

During pipeline execution, programmatic set assertions are executed before any file is serialized to disk:

```
# Aggregate all reserved evaluation IDs
test_ids = set(ids_100) | set(ids_1000)

# Verify set disjointness across splits
assert ids_100.isdisjoint(ids_1000), "Overlap detected between test_100 and test_1000!"
assert set(kb_10k).isdisjoint(test_ids), "Data leakage! Knowledge base contains test
set IDs."
assert set(val_100).isdisjoint(test_ids), "Data leakage! Validation set contains test
set IDs."
```

8- Repository Workflow & Dataset Freezing Rules

To prevent silent corruption of splits during multi-developer collaboration, strict repository governance rules are maintained:

- **Dataset Freezing:** Files under `phase_01_data_exploration/splitted_data_set/` are locked. Only the repository lead (`rafie`) is authorized to modify frozen splits.
- **Branch Isolation:** Developers work on dedicated feature branches (`bella`, `hadil`, `raouf`, `zinou`) and merge into `main` via pull requests only after regression testing.

9- Statistical EDA & Hybrid Retrieval Architecture Design

Exploratory Data Analysis (EDA) is not just a reporting step, it directly informs key architectural choices for the downstream RAG retriever and VLM generator.

1- Object Class Distribution & Long-Tail Hallucination Risk

Mapping instance annotations across the 80 COCO categories reveals a heavy long-tailed distribution. The `person` category ranks #1 across all four sub-datasets (e.g., 17,177 instances in `kb_10k.json`), occurring over **2.5 times more frequently** than the second-ranked category (`car`, 6,820 instances).

Architectural Impact: Un-augmented VLMs naturally develop strong visual priors for dominant classes like `person` and `car`, causing them to hallucinate these terms when facing noisy inputs. However, `kb_10k.json` holds dense occurrence clusters for these primary classes alongside long-tail objects, providing the dense embedding space required for vector retrieval.

2- Scene Complexity & Outlier Resilience

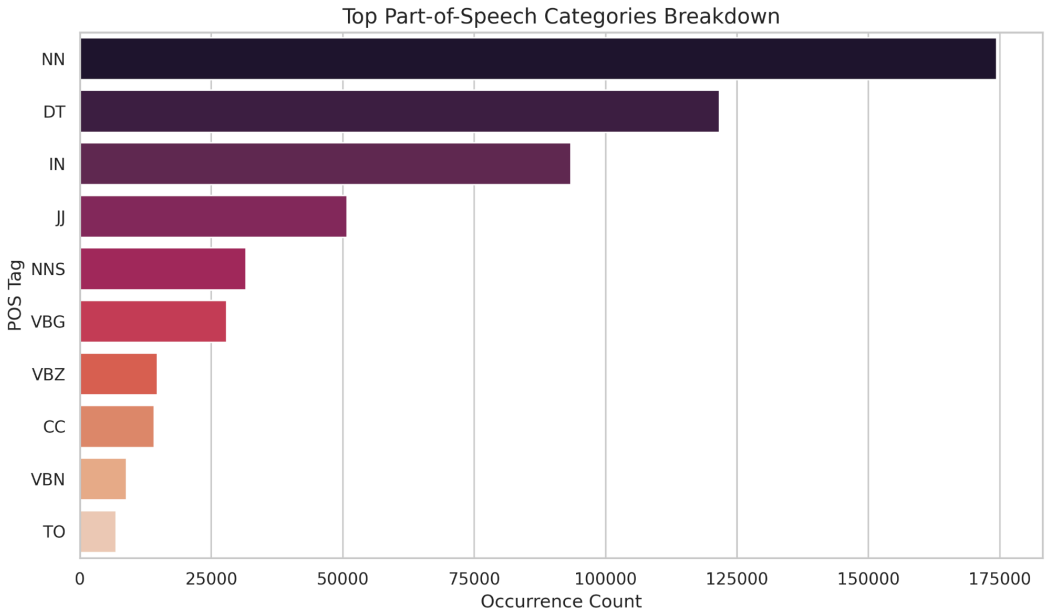
In long-tailed object counts per image, massive outliers (such as crowded market stalls containing 50+ items) heavily inflate the **Mean** (e.g., 6.71 objects/image in `kb_10k.json`). The **Median** (4.00 objects/image) serves as a far more resilient metric of central tendency.

Architectural Impact: Thanks to stratification, both `test_100.json` and `test_1000.json` exhibit an exact **50.00% / 50.00% split** between simple ($1 - 2$ objects) and complex (≥ 3 objects) scenes. This guarantees that prompt engineering and retrieval tuning executed on small benchmarks transfer directly to large-scale testing.

3- Syntactic Part-of-Speech (POS) Distribution

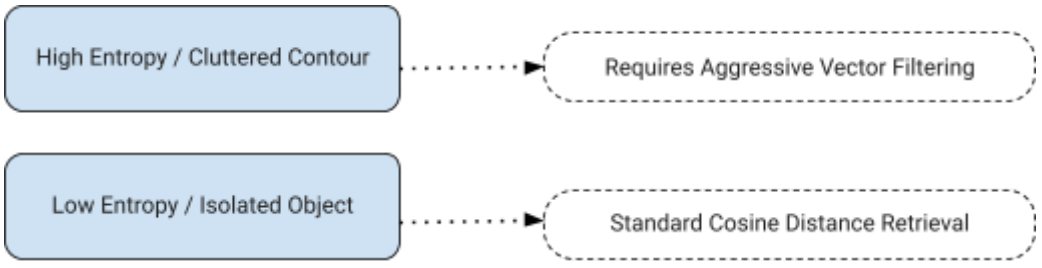
Tokenizing captions with NLTK reveals extraordinary lexical consistency: all splits maintain a median caption length of exactly **10.00 words**. Part-of-Speech tagging across `kb_10k.json` shows:

- **Nouns (NN):** 35.45% (Object identification)
- **Determiners (DT):** 21.44% (Noun phrase bounding)
- **Prepositions (IN):** 16.05% (Spatial relationships: *on, under, behind*)
- **Adjectives (JJ):** 8.85% (Visual attributes: *red, wooden, large)*



Downstream Hybrid Indexing Strategy
To maximize visual grounding, these statistical insights dictate a **Dual-Index Hybrid Retrieval System**:

1. **Dense Visual Indexing (CLIP / SBERT):** Vectorizes visual and textual semantic representations, capturing spatial prepositional relationships and adjective-noun attribute pairings.
2. **Sparse Lexical Indexing (BM25 / TF-IDF):** Indexes the exact 9,162 unique vocabulary terms of `kb_10k.json`, ensuring rare long-tail object categories (e.g., `zebra`, `hair dryer`) missed by dense semantic search are accurately retrieved.



Visual Shape Complexity vs. Spatial Density

Quantifying these visual and structural data distributions establishes the baseline specifications required for strict dataset partitioning and leakage-proof split ingestion.

10. Dataset Partitioning, Anti-Leakage Governance, and Ingestion Audit

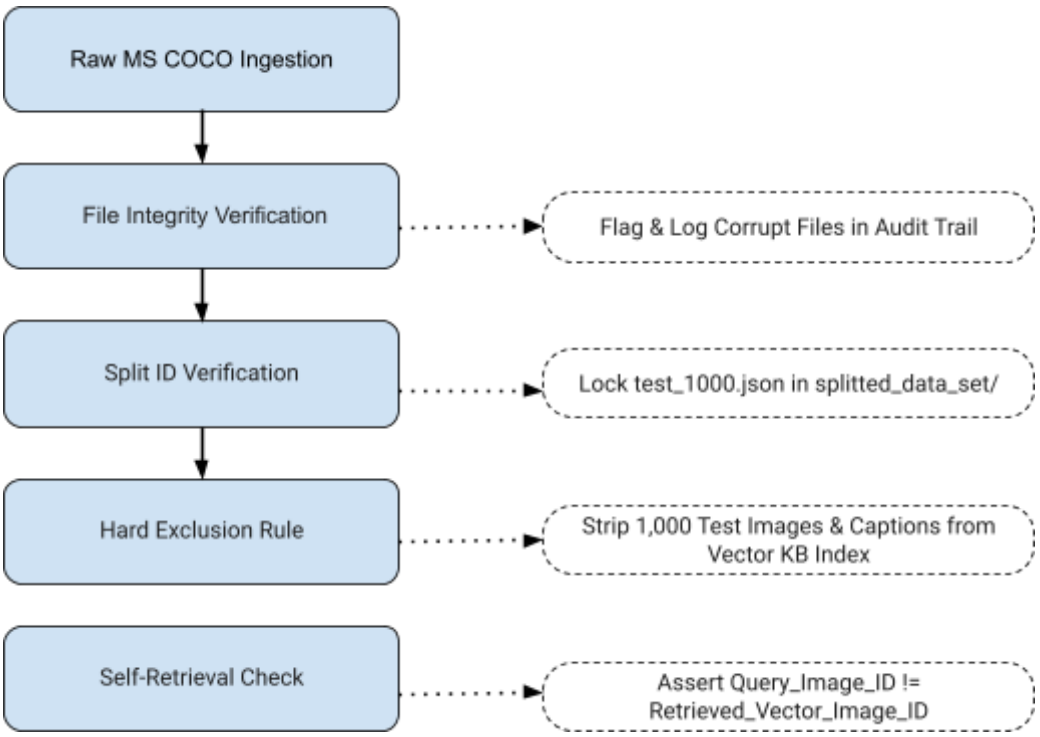
In a retrieval-augmented generation framework, absolute split isolation is an operational requirement. If reference captions or visual embeddings belonging to test samples leak into the vector knowledge base, the retrieval module will perform self-retrieval (matching query test samples against their own ground-truth data). This invalidates automated evaluation metrics (CIDEr, SPICE, BLEU) and masks underlying VLM hallucination tendencies.

Ingestion Inputs & Dataset Split Specifications

Environmental Dimension	Architectural Diagnostic Impact
Indoor vs. Outdoor Contexts	Shifts spatial object frequency and background noise levels.
Day vs. Night Illumination	Alters contrast ratios and deep visual feature extraction.
Color Distributions (RGB / HSV)	Induces chromatic bias in visual backbone activations.
Visual Shape Complexity	High geometric clutter increases VLM hallucination rates.

Audit & Anti-Leakage Protocol Execution

Ingestion auditing executes a deterministic validation protocol prior to vector construction:



1. **File Integrity Verification:** Ingested image files and JSON metadata files are checked for unreadable bitstreams or truncated structural trees. Any corrupted file is excluded, with actions logged to an audit trail.
2. **Versioned Frozen Split Storage:** Target split configurations are saved as locked JSON files under `phase_01_data_exploration/splitted_data_set/` (e.g., `test_1000.json`).
3. **Strict Anti-Leakage Control Enforcement:**
    - **Index Exclusion Gate:** Reference captions and visual feature vectors associated with the 1,000 test evaluation samples (`test_1000.json`) must be blocked from entering the vector database index.
    - **Self-Retrieval Query Filter:** The retrieval engine executes an explicit filter condition during k-NN vector matching, asserting that $Image_ID_{query} \neq Image_ID_{retrieved}$.

Enforcing absolute partition isolation enables raw image tensors and text sequences to be ingested into the dual-track preprocessing engine safely.

11. Multimodal Preprocessing & Metadata Extraction Engine

Raw MS COCO image resolutions, variable aspect ratios, spatial orientations, and noisy caption strings are incompatible with vector operations. The preprocessing engine transforms heterogeneous inputs into standardized tensor formats and isolated text channels required by dual encoders and structured metadata indexes.

Image Preprocessing Pipeline

Visual preprocessing applies deterministic spatial and color normalizations to raw images prior to feature extraction:

1. **Color Format Uniformity:** Standardizes all input images to 3-channel RGB format, stripping alpha channels or grayscale profiles.

2. **EXIF Orientation Correction:** Checks and applies EXIF orientation metadata flags, eliminating rotational mismatches.
3. **Aspect-Ratio Preserved Resizing:** Resizes images to encoder dimensions (e.g., 224×224 or 336×336 pixels) using bicubic interpolation combined with reflective padding to prevent spatial distortion.
4. **Tensor Normalization:** Standardizes pixel intensities across RGB channels using ImageNet mean ($\mu = [0.485, 0.456, 0.406]$) and standard deviation ($\sigma = [0.229, 0.224, 0.225]$) parameters.
5. **Object Detection Metadata Harvesting:** Executes an optional object detection pass using Faster R-CNN (ResNet-50-FPN) or DINO to extract localized bounding boxes, object category labels, and region confidence scores for metadata logging.

Dual-Track Caption Preprocessing Architecture

Textual captions are ingested through two isolated transformation pipelines to separate retrieval operations from metric evaluations:

Processing Parameter	Raw Reference Captions Track	Cleaned Captions Track
System Purpose	Ground-truth benchmark for automated metrics (CIDEr, SPICE, BLEU-4).	Vector embedding generation, indexing, and k-NN semantic search.
Text Normalization	Zero transformations. Case, punctuation, and contractions are preserved.	Lowercased, ASCII-normalized, punctuation removed, excess whitespace collapsed.
Field Name Isolation	Bound to JSON field: `raw_caption`	Bound to JSON field: `clean_caption`
Token Filtering	Original reference tokens maintained without modification.	Optional stop-word filtering applied for dense vector indexing.
Pipeline Storage	Stored in static evaluation metadata tables.	Processed by text embedding encoders for vector database creation.

Structured Metadata Extraction Schema

Harvested visual detection records, attribute tags, scene tags, and captions are serialized into a uniform JSON schema linked by a primary key (`image_id`).

```
{
  "image_id": 242158,
  "split_id": "train_10000",
  "image_dimensions": {
    "height": 480,
    "width": 640,
    "channels": 3
  },
  "raw_captions": [
    "A dog catching a red frisbee in a sunny park.",
    "A black lab jumps to catch a flying disc outdoors."
  ],
  "clean_captions": [
    "dog catching red frisbee sunny park",
    "black lab jumps catch flying disc outdoors"
  ],
  "detected_objects": [
    {
      "object_id": 1,
      "label": "dog",
      "bounding_box": [120.5, 45.2, 210.0, 315.8],
      "detection_confidence_threshold": 0.942
    },
    {
      "object_id": 2,
      "label": "frisbee",
      "bounding_box": [310.2, 88.0, 45.0, 42.1],
      "detection_confidence_threshold": 0.887
    }
  ],
  "detector_model_version": "FasterRCNN_ResNet50_FPN_v2.1",
  "attributes": ["outdoor", "daylight", "grassy"],
  "embedding_path": "embeddings/train_10000/img_242158.npy"
}
```


Standardizing multimodal artifacts into concrete schemas allows downstream dense visual and textual encoders to generate reproducible vector indices.

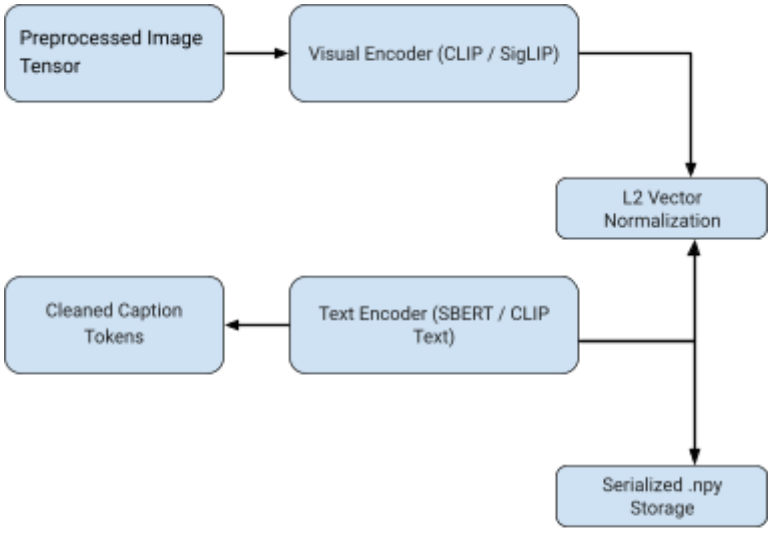
---

12. Multimodal Embedding Generation & Vector Knowledge-Base Indexing

Transforming visual attributes and text captions into dense continuous vector spaces enables sub-linear nearest-neighbor retrieval. The vector knowledge-base architecture must balance retrieval recall, search latency, and memory footprint. This section defines the dual-encoder embedding extraction pipeline and vector index topology.

Embedding Generation Pipeline

The pipeline supports multiple vision-language projection backbones for generating multi-modal representations:



DUAL-ENCODER EMBEDDING PIPELINE

- **Visual Encoders:**
  - **CLIP (Vision Transformer - ViT-B/32 & ViT-L/14):** Maps image frames to 512-dimensional ($d = 512$) or 768-dimensional ($d = 768$) embedding spaces.
  - **SigLIP (SigLIP-So400M):** Extracts high-density visual vectors ($d = 1152$) optimized for fine-grained image-text alignment.
- **Text Encoders:**
  - **Sentence-BERT (SBERT all-MiniLM-L6-v2 / all-mpnet-base-v2):** Generates 384-d or 768-d semantically dense text representations.
  - **CLIP Text Encoder:** Maps text captions into a shared joint embedding space with visual feature vectors.
- **Execution Parameters & Serialization:**
  - **Batch Processing:** Feature extraction runs at fixed batch sizes ($N = 64$ or $N = 128$) using GPU acceleration.
  - L_2 **Vector Normalization:** All dense vectors are normalized to unit length: $$\hat{v} = \frac{v}{\|v\|_2}$$
  - **Output Serialization:** Extracted embeddings are serialized as float32 NumPy arrays (`.npy`) on disk, mapped directly to their respective `image_id` index.

Vector Database & Knowledge-Base Construction

The system evaluates three vector storage engines: FAISS, ChromaDB, and LanceDB.

Vector Engine	Search Speed / Latency	Index Scalability	Metadata Filtering
FAISS	Ultra-Low (< 2ms k-NN query)	In-Memory / Billion	In-Memory Bitset Map

ChromaDB	Low (< 15ms k-NN query)	Persistent / Local	Native JSON Payload
LanceDB	Low-Medium (< 10ms k-NN query)	On-Disk Columnar	Embedded Arrow Queries

For production vector retrieval, **FAISS (Facebook AI Similarity Search)** is designated as the primary engine due to its low retrieval latency during dense vector indexing.

Index Topologies & Distance Metrics

The vector database architecture supports three configurable index topologies:

1. **Flat Index (IndexFlatIP / IndexFlatL2):** Exhaustive exact nearest-neighbor search. Serves as the zero-quantization baseline.
2. **Inverted File Index (IndexIVFFlat):** Partitioned vector space utilizing Voronoi cells ($n_{list} = 100$, $n_{probe} = 10$). Accelerates search speed by limiting distance calculations to adjacent centroids.
3. **Hierarchical Navigable Small World (IndexHNSWFlat):** Graph-based indexing ($M = 32$ connections per node, $ef_{Construction} = 200$, $ef_{Search} = 64$). Delivers sub-linear retrieval speed with high query recall (> 98%).

Vector distance logic uses Cosine Similarity, Inner Product (IP), and Euclidean Distance (L_2):

$$\text{Cosine Similarity: } \cos(\theta) = \frac{\mathbf{q} \cdot \mathbf{v}}{\|\mathbf{q}\|_2 \|\mathbf{v}\|_2}$$

$$InnerProduct(normalizedvectors): D_{IP}(q, v) = q \cdot v$$

$$EuclideanDistance: D_{L2}(q, v) = \sqrt{\sum_{i=1}^{d} (q_i - v_i)^2}$$

System Performance & Logging Metrics

Vector index construction pipelines systematically capture and log system runtime parameters:

- **Vector Dimension Size (d):** Explicitly logged (e.g., $d = 512$, $d = 768$).
- **Memory Footprint:** Measured in megabytes (MB) or gigabytes (GB) across built indices.
- **Index Build Latency:** Total CPU/GPU wall-clock build time recorded in seconds.
- **Retrieval Query Latency:** Average milliseconds (ms) per k-NN lookup across test queries.

Extracted vector indices are bound to structured JSON metadata, facilitating downstream automated validation and release freezing.

13. Automated Quality Control, System Validation, and Immutable Freezing

Automated Quality Control (QC) gates prevent technical debt, vector drift, and silent data corruption across distributed branches . Automated verification ensures that only baseline-validated code and data structures are merged into `main`.

Comprehensive Quality Control (QC) Matrix

Prior to opening a PR into `main`, contributors must execute an automated verification script that evaluates the system state against this checklist:

Validation Task	Verification Criteria
Embedding Completeness Audit	Zero missing vectors; count matches input split IDs.
Numerical Integrity Check	Zero NaN, Null, or Inf values across serialized `.npy` files.
ID Alignment Verification	Strict 1:1 mapping between Image ID, JSON, and Vectors.
Caption Mismatch Verification	Length of `raw_captions` == `clean_captions` for all records.
Duplicate Entry Audit	Zero duplicate Image IDs or vector key collisions.
Anti-Leakage Verification Gate	Zero `test_1000.json` entries in active vector index.
Pre-Merge Statistical Summaries	POS distributions, vocabulary coverage, and complexity metrics calculated and appended to PR logs.
Sample Visualization Audit	Diagnostic plots (t-SNE/UMAP) checked and logged.

Notebook Top-to-Bottom Execution	All notebooks execute sequentially without runtime errors.

Validation Subsets & Freezing Protocol

Operational sign-off relies on isolated validation gates:

1. **Role of the 100-Image Validation Subset:** The frozen 100-image validation subset acts as an isolated parameter tuning bed. Preprocessing options (e.g., token pruning strategies, spatial padding choices, vector distance metrics) are evaluated on this subset to select optimal settings without touching or corrupting the downstream 1,000-image evaluation set (`test_1000.json`).
2. **Immutable Release Freezing Rule:** Once optimal preprocessing parameters are confirmed, output artifacts situated under `phase_01_data_exploration/splitted_data_set/` are locked.
3. **Data Freezing Update Protocol:** If the lead branch (`rafie`) modifies a frozen split file, the Mandatory Re-run Protocol is triggered: all ingestion, vector generation, and QC verification steps across all project modules must be re-executed, and updated log files must be committed to the PR prior to merging into `main`.

Adherence to this architectural specification guarantees a reproducible, hallucination-resistant RAG framework that grounds Vision-Language Models through dynamic context compression.

Summary

Constructing a grounded Multimodal RAG system requires strict engineering controls at the data acquisition and partitioning stage. By combining multi-dimensional stratified sampling, dynamic fallback reallocation, programmatic zero-leakage assertions, and EDA-guided hybrid indexing, we eliminate experimental bias and equip VLMs to eliminate object hallucinations.

Key Takeaways

- **Candidate Filtering:** Validating physical disk integrity and truncating human reference captions to exactly 5 per image ensures evaluation consistency.
- **Multi-Dimensional Stratification:** Classifying scenes across Complexity, Interaction, and Context prevents benchmark skew; dynamic reallocation handles real-world class starvation seamlessly.
- **Zero-Leakage Assertions:** Enforcing programmatic set disjointness (`isdisjoint()`) guarantees complete isolation between retrieval knowledge bases (`kb_10k.json`) and test splits (`test_1000.json`).
- **Hybrid Index Design:** High noun density (35.5%) and long-tail category imbalance demand a dual retrieval setup: Dense visual embeddings (CLIP) for semantic context, paired with Sparse lexical indexing (BM25) for long-tail vocabulary precision.